

SIMATS ENGINEERING ASSESSMENT TOOL 3

NAME: MURALI KRISHNAN N

REG NO: 192524090

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0613

COURSE OUTCOME COVERED

CO 4 : Solve optimization problems using dynamic programming and greedy strategies.

Levels of Rubrics

(BL-4, BL-5)

Annexure A

Comparative Analysis

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive

Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Q25. Dice Throw Problem vs Binomial Coefficient (DP Comparison)

Question (as given in the Assessment Tool): Compute the number of ways to achieve a sum using dice and compare it with computing binomial coefficients. Analyze similarities in DP table construction. Identify differences in recurrence relations. Evaluate computational complexity. Discuss memory optimization. Generate insights on combinatorial DP problems.

1. Problem Statement — Dice Throw Problem

Given m identical dice, each with n faces numbered 1 to n , the Dice Throw Problem asks: in how many distinct ways can the dice be thrown so that the sum of the numbers shown equals a given target value S ? Two throws are considered different arrangements if the sequence of face values differs, even if the multiset of values is the same (i.e., order across dice matters, since each die is distinguishable by position).

This is an example of a combinatorial counting problem with overlapping subproblems: the number of ways to reach sum S using m dice depends only on the number of ways to reach smaller sums using $m-1$ dice, so the same subproblem (a given number of dice and a given remaining sum) recurs many times across different branches of the naive recursive solution — which is exactly what makes Dynamic Programming applicable.

Illustrative instance used throughout this answer: $m = 2$ dice, $n = 6$ faces, target sum $S = 7$.

2. Recurrence Relation — Dice Throw

Let $dp[i][j]$ denote the number of ways to obtain a sum of j using exactly i dice. A single die can show any face value f in the range 1 to n , so the count for i dice and sum j is obtained by summing, over every possible value f shown on the i -th die, the number of ways the remaining $(i-1)$ dice can make up the balance $(j - f)$:

$$dp[i][j] = \sum dp[i-1][j-f] \quad \text{for } f = 1 \text{ to } \min(n, j)$$

Base case: $dp[0][0] = 1$ (there is exactly one way to make a sum of zero using zero dice — throw nothing), and $dp[0][j] = 0$ for all $j > 0$ (a sum greater than zero is impossible with no dice).

3. Algorithm (Pseudocode) — Dice Throw

```
ALGORITHM DiceThrowWays(m, n, S)
  Input : m - number of dice
          n - number of faces per die
          S - target sum
  Output: number of distinct ways to reach sum S

  create table dp[0..m][0..S], initialise all entries to 0
  dp[0][0] = 1

  for i = 1 to m:
    for j = 0 to S:
      for f = 1 to min(n, j):
        dp[i][j] = dp[i][j] + dp[i-1][j-f]

  return dp[m][S]
```

The three nested loops make the time complexity $O(m \cdot S \cdot n)$, and the table itself occupies $O(m \cdot S)$ space.

4. Python Implementation — Dice Throw

```

def dice_throw_ways(m, n, target):
    """
    Count the number of ways to get 'target' as the sum
    of m dice, each having n faces (values 1..n).
    """
    dp = [[0] * (target + 1) for _ in range(m + 1)]
    dp[0][0] = 1 # base case

    for dice in range(1, m + 1):
        for s in range(0, target + 1):
            for face in range(1, min(n, s) + 1):
                dp[dice][s] += dp[dice - 1][s - face]

    return dp[m][target]

if __name__ == "__main__":
    m, n = 2, 6 # 2 dice, 6 faces each
    for target in range(2, 13): # all reachable sums
        ways = dice_throw_ways(m, n, target)
        print(f"Sum = {target:2d} -> Ways = {ways}")

```

Program Output (2 dice, 6 faces each):

Sum (S)	2	3	4	5	6	7	8	9	10	11	12
Ways	1	2	3	4	5	6	5	4	3	2	1

For the target used in the question setup ($S = 7$, $m = 2$, $n = 6$), the program returns 6 ways: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1) — matching the well-known symmetric distribution of a two-dice sum, which peaks at 7 and tapers off toward the extremes 2 and 12.

5. Problem Statement — Binomial Coefficient

The Binomial Coefficient $C(n, k)$ counts the number of ways to choose an unordered subset of k elements from a set of n distinct elements, without repetition. It is defined mathematically as $n! / (k! \cdot (n-k)!)$, but computing it directly via factorials is prone to overflow for large n and performs redundant multiplication. Pascal's Identity gives a recursive, addition-only alternative:

$$C(n, k) = C(n-1, k-1) + C(n-1, k)$$

with base cases $C(n, 0) = 1$ and $C(n, n) = 1$. As with the dice throw problem, the naive recursive expansion of this identity revisits the same (n, k) pairs repeatedly, giving exponential-time recursion unless the overlapping subproblems are cached — which is precisely the role Dynamic Programming plays here.

6. Recurrence Relation and Algorithm (Pseudocode) — Binomial Coefficient

```

ALGORITHM BinomialCoefficient(n, k)
    Input : n, k  (0 ≤ k ≤ n)

```

```

Output: C(n, k)

create table dp[0..n][0..k], initialise all entries to 0

for i = 0 to n:
    for j = 0 to min(i, k):
        if j == 0 or j == i:
            dp[i][j] = 1                                // base case
        else:
            dp[i][j] = dp[i-1][j-1] + dp[i-1][j]

return dp[n][k]

```

This is the classical Pascal's Triangle construction: each entry is the sum of the two entries diagonally above it in the previous row. The nested loop structure gives a time complexity of $O(n \cdot k)$ and a space complexity of $O(n \cdot k)$, which is reducible to $O(k)$ with a single rolling 1-D array.

7. Python Implementation — Binomial Coefficient

```

def binomial_coefficient(n, k):
    """
    Compute C(n, k) using Pascal's Identity via
    bottom-up Dynamic Programming (Pascal's Triangle).
    """
    dp = [[0] * (k + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        for j in range(min(i, k) + 1):
            if j == 0 or j == i:
                dp[i][j] = 1
            else:
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]

    return dp[n][k]

def binomial_coefficient_1d(n, k):
    """Space-optimised O(k) version using a rolling array."""
    dp = [0] * (k + 1)
    dp[0] = 1
    for i in range(1, n + 1):
        for j in range(min(i, k), 0, -1): # iterate right-to-left
            dp[j] += dp[j - 1]
    return dp[k]

if __name__ == "__main__":
    n = 6
    print(f"Pascal's Triangle up to row {n}:")
    for i in range(n + 1):
        row = [binomial_coefficient(i, j) for j in range(i + 1)]
        print(row)

```

```
print("C(5, 2) =", binomial_coefficient(5, 2))
print("C(5, 2) [1-D version] =", binomial_coefficient_1d(5, 2))
```

Program Output — Pascal's Triangle (rows 0 to 6):

```
Row 0: [1]
Row 1: [1, 1]
Row 2: [1, 2, 1]
Row 3: [1, 3, 3, 1]
Row 4: [1, 4, 6, 4, 1]
Row 5: [1, 5, 10, 10, 5, 1]
Row 6: [1, 6, 15, 20, 15, 6, 1]

C(5, 2) = 10
C(5, 2) [1-D version] = 10
```

Both the 2-D table version and the space-optimised 1-D rolling-array version agree on $C(5, 2) = 10$, confirming that the $O(k)$ memory reduction (iterating j from high to low so each cell is updated using values not yet overwritten in the current pass) does not change correctness.

7A. Manual Dry Run — Dice Throw DP Table

To make the table-filling process concrete, the dp table for $m = 2$ dice, $n = 6$ faces is traced below for sums 0 through 8 (a truncated view; the full table extends to sum = 12). Row $i = 1$ shows the ways to reach each sum with a single die (1 way each, for sums 1–6); row $i = 2$ shows the ways to reach each sum with two dice, computed by summing the previous row over the last 6 columns.

dp[i][j]	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7	j=8
i = 0	1	0	0	0	0	0	0	0	0
i = 1	0	1	1	1	1	1	1	0	0
i = 2	0	0	1	2	3	4	5	6	5

Reading $dp[2][7] = 6$ confirms the earlier program output: with two 6-faced dice there are exactly six ordered combinations that sum to 7. Each entry in row $i=2$ is computed as the sum of up to six entries directly above it and to its left in row $i=1$, exactly as the recurrence in Section 2 specifies.

7B. Manual Dry Run — Binomial Coefficient DP Table

The corresponding dry run for $C(n, k)$ up to $n = 5$ reproduces Pascal's Triangle. Each interior entry is the sum of exactly two entries from the row above — the cell directly above it and the cell diagonally above-left — never more than two, regardless of how large n or k grows.

dp[i][j]	j=0	j=1	j=2	j=3	j=4	j=5
i = 0	1	-	-	-	-	-
i = 1	1	1	-	-	-	-

dp[i][j]	j=0	j=1	j=2	j=3	j=4	j=5
i = 2	1	2	1	-	-	-
i = 3	1	3	3	1	-	-
i = 4	1	4	6	4	1	-
i = 5	1	5	10	10	5	1

Reading $dp[5][2] = 10$ confirms $C(5, 2) = 10$, matching both the Python outputs shown in Section 7. Placing the two dry-run tables side by side highlights the structural point made in Section 9: the dice-throw table's rows widen their 'reach' by up to six columns at a time, while the binomial table's rows always reach back by exactly one and two columns — a direct visualisation of the branching-factor difference between the two recurrences.

7C. Graph-of-Growth: How the Branching Factor Affects Table-Filling Cost

The number of arithmetic operations performed while filling one row of each table follows directly from the branching factor discussed above. For Dice Throw with an n -faced die, filling one cell requires up to n additions, so one full row of $S+1$ cells costs $O(n \cdot S)$ additions; across m dice this gives the stated $O(m \cdot n \cdot S)$ total. For Binomial Coefficient, filling one cell always requires exactly one addition ($dp[i-1][j-1] + dp[i-1][j]$), so one full row of $k+1$ cells costs $O(k)$ additions, giving $O(n \cdot k)$ total across n rows.

This means that, for a fixed table size, Dice Throw's running time grows linearly with the number of faces n even though n does not appear as a table dimension — it is a hidden multiplicative constant inside the recurrence's inner loop. This is an important practical lesson: two DP algorithms with tables of the same shape (rows $\times$ columns) need not have the same asymptotic running time if their per-cell recurrence work differs.

7D. Optimised Dice Throw Using Prefix Sums (Reducing the Hidden Factor of n)

The insight generated in Section 10 — that Dice Throw's per-cell cost can be reduced from $O(n)$ to $O(1)$ — is demonstrated below using a prefix-sum (sliding window) technique. Instead of re-summing up to n previous cells for every new cell, a running prefix sum of the previous row is maintained and updated incrementally as j increases, so each new cell is computed with one addition and one subtraction regardless of n .

```
def dice_throw_ways_optimised(m, n, target):
    """
    O(m * S) version of Dice Throw using a prefix-sum
    (sliding window) over the previous row, removing the
    inner loop over face values (1..n).
    """
    dp = [[0] * (target + 1) for _ in range(m + 1)]
    dp[0][0] = 1

    for dice in range(1, m + 1):
        prefix = 0
        # running window sum
```

```

    for s in range(0, target + 1):
        prefix += dp[dice - 1][s]
        if s - n >= 0:
            prefix -= dp[dice - 1][s - n]
        dp[dice][s] = prefix if s > 0 else 0

    return dp[m][target]

if __name__ == "__main__":
    print(dice_throw_ways_optimised(2, 6, 7))    # -> 6
    print(dice_throw_ways_optimised(2, 6, 10))   # -> 3

```

This optimisation brings Dice Throw's overall time complexity down to $O(m \cdot S)$, the same shape as Binomial Coefficient's $O(n \cdot k)$ — reinforcing the insight that once the branching factor is isolated into an explicit loop, standard techniques (here, a sliding-window prefix sum) can often eliminate it, bringing two superficially different counting-DP problems to matching asymptotic complexity.

8. Side-by-Side Comparison

Parameter	Dice Throw DP	Binomial Coefficient DP
Counts	Number of ways to reach a target sum using m dice	Number of ways to choose k items from n
State (table axes)	<code>dp[dice][sum]</code>	<code>dp[n][k]</code>
Recurrence	<code>dp[i][j] = sum of dp[i-1][j-f] for f = 1..min(n,j)</code>	<code>dp[i][j] = dp[i-1][j-1] + dp[i-1][j]</code>
Terms combined per cell	Up to n terms (one per face value)	Exactly 2 terms (include / exclude)
Base case	<code>dp[0][0] = 1, dp[0][j>0] = 0</code>	<code>dp[i][0] = 1, dp[i][i] = 1</code>
Time complexity	$O(m \cdot S \cdot n)$	$O(n \cdot k)$
Space complexity (naive)	$O(m \cdot S)$	$O(n \cdot k)$
Space complexity (optimised)	$O(S)$, one row at a time	$O(k)$, one rolling array
Branching factor	Variable — equal to the number of faces n	Fixed — always 2 (binary choice)
Table-filling direction	Row-by-row over dice count, then sum	Row-by-row over n, then column j

9. Critical Evaluation

Both problems are solved with a bottom-up tabulation DP that fills a two-dimensional grid, where each cell is derived purely from cells in the previous row — a signature of overlapping subproblems and optimal substructure. This structural similarity means both are natural candidates for the same class of implementation techniques: iterative (not recursive) table filling, and rolling-array space optimisation, since neither recurrence needs more than the immediately preceding row to compute the current one.

The essential difference lies in the branching factor of the recurrence — that is, how many terms from the previous row are combined to produce one cell in the current row. Binomial Coefficient always combines exactly two terms ($dp[i-1][j-1]$ and $dp[i-1][j]$), reflecting the binary include/exclude decision inherent to subset selection. Dice Throw combines up to n terms (one for every possible face value of the current die), reflecting the fact that a single die roll is not a binary decision but a choice among n outcomes. This is why the Dice Throw recurrence has an inner loop over face values that the Binomial Coefficient recurrence does not need.

This difference also drives the time-complexity gap: Binomial Coefficient's $O(n \cdot k)$ is strictly a function of the two dimensions of the table, whereas Dice Throw's $O(m \cdot S \cdot n)$ carries an extra multiplicative factor of n because every cell requires an internal loop over face values. If the dice had only 2 faces (a coin), the dice-throw recurrence would degenerate to combining exactly 2 terms per cell — structurally identical in shape to the binomial recurrence, though not identical in meaning.

From a memory-optimisation standpoint, both DP tables can be compressed from $O(\text{rows} \times \text{columns})$ to $O(\text{columns})$ using a single 1-D array updated in place, provided the iteration order respects data dependencies. For Binomial Coefficient, this requires iterating the inner index j from high to low within each outer pass (as shown in `binomial_coefficient_1d` above) so that $dp[j-1]$ read during the update still holds the previous row's value and has not yet been overwritten by the current row. An equivalent left-to-right vs right-to-left care is required for Dice Throw if a 1-D optimisation is attempted, because $dp[i][j]$ depends on multiple previous-row cells $dp[i-1][j-1]$ through $dp[i-1][j-n]$, not just one.

10. Synthesis and Insight Generation

- Both problems belong to a broader family of 'counting DP' problems where a table cell aggregates a bounded number of earlier cells; identifying a new problem's branching factor (2, n , or otherwise) is the fastest way to adapt an existing DP template to it.
- A recurrence's inner loop bound is not an arbitrary implementation detail — it directly encodes the number of outcomes available at each decision point in the real-world problem (2 outcomes for include/exclude, n outcomes for a die face).
- Space optimisation from a 2-D table to a 1-D rolling array is possible whenever a recurrence depends only on the immediately preceding row; the direction of iteration (forward or reverse) must be chosen so that a cell is never overwritten before it is needed, which differs between the two problems and must be verified case-by-case.
- Because Dice Throw's per-cell cost scales with the number of faces n , it is more sensitive to input parameters than Binomial Coefficient; for large n (many-sided dice), a further optimisation using prefix sums can reduce Dice Throw's inner loop from $O(n)$ to $O(1)$ per cell, bringing its overall complexity down to $O(m \cdot S)$, matching the shape of Binomial Coefficient's complexity.
- Both problems illustrate that Dynamic Programming's core value is reuse of previously computed subproblem answers; the choice of DP over plain recursion is justified whenever the same (state) pair — (dice-used, sum) or (n, k) — would otherwise be recomputed many times.

10A. Real-World Applications

Dice Throw–style counting DP appears wherever a fixed number of independent, discrete-valued trials must be combined to hit a target total: dice and board games, probability distributions for tabletop RPG damage rolls, network packet-loss modelling with a bounded per-slot loss count, and Monte Carlo–adjacent exact-probability calculations where an exact count is preferred over simulation.

Binomial Coefficient DP underlies an even broader range of applications: computing probabilities in the binomial distribution, counting the number of paths in a grid, combinatorial subset-selection problems (choosing committees, choosing test cases, feature subsets in machine learning), and as a building block inside more advanced DP formulations such as the Knapsack problem's counting variant and certain formulations of Catalan numbers.

11. Conclusion

Dice Throw and Binomial Coefficient are structurally related counting problems solved by the same class of bottom-up tabulation DP, differing chiefly in their branching factor: a fixed binary choice for Binomial Coefficient versus a variable, face-count-dependent choice for Dice Throw. Recognising this shared 'aggregate a bounded set of prior-row cells' template — and knowing which structural parameter (branching factor) governs the extra cost — allows a DP solution designed for one combinatorial counting problem to be adapted quickly and correctly to a new one, while also indicating where further optimisations (such as prefix sums) can close the complexity gap between them.